Reviewer Pack — Spectral-Tail Mass of Unit-Propagation Indicator Bounds DPLL...

Entry 9edcbf300e22 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-27 23:55:31 UTC

Spectral-Tail Mass of Unit-Propagation Indicator Bounds DPLL Leaves

Entry ID: `9edcbf300e22` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-04-27 23:55:31 UTC

1. Conjecture

Field A (mathematical branch): Fourier analysis of Boolean functions: high-degree Walsh spectrum tail mass W^{>k}[g] = sum_{|S|>k} hat g(S)^2 of the unit-propagation success indicator **Field B** (complexity object): DPLL search-tree leaf count L(F) on small unsatisfiable 3-CNF formulas under a fixed lex variable order with unit propagation

Statement:

> For an unsatisfiable 3-CNF F on n variables with m clauses, define g_F:{-1,+1}^n -> {0,1} by g_F(x)=1 iff iterated unit propagation on F under the partial assignment x reaches the empty clause. Let T_k(F) = sum_{|S|>k} hat g_F(S)^2 be the degree->k Walsh tail. Then for every unsatisfiable 3-CNF with n<=20 and m<=5n, log2(L(F)+1) <= 4 * (1 + T_{log2(m+1)}(F) * 2^n) / max(1, hat g_F(emptyset) * 2^n), and for at least 60% of random unsatisfiable instances at clause density 4.5 the ratio (log2(L(F)+1)) / (1 + T_{log2(m+1)}(F) * 2^n / max(1, hat g_F(emptyset) * 2^n)) lies in [0.05, 4].

Rationale (proposer's reasoning):

> Unit propagation is a low-degree Boolean process: each propagation step depends on at most 3 literals, so the indicator g_F should have most of its Fourier mass on low-degree coefficients when F is locally easy, and DPLL backtracks exactly where g_F's high-degree tail concentrates (KKL/Friedgut intuition: rare, dispersed sensitivities force junta failure). Quantifying the high-degree mass of g_F therefore offers a hypercontractivity-flavored explanation for DPLL leaf blow-up that, unlike total influence on the SAT indicator, is computable directly from F by simulating UP on every assignment.

Taxonomy category: `FOURIER_ANALYTIC` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `fa0445e8ece1cd46`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across >=30 UNSAT 3-CNFs per n in {8,10,12,14} (m=floor(4.5n)) plus PHP_3 and 5 near-threshold hard instances, every instance must satisfy log2(L+1) <= 4*(1+T_k2^n)/max(1,ĝ(□)*2^n) with k=floor(log2(m+1)), AND >=60% of random instances must have ratio in [0.05,4].

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.92	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.92	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.96	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `Fourier analysis Boolean functions unit propagation Walsh spectrum 3-CNF` - `DPLL search tree size Walsh spectrum tail unsatisfiable 3-SAT complexity` - `Fourier degree concentration SAT solver propagation indicator function lower bound`

Top relevant hits considered: - [http://arxiv.org/abs/1504.04131v2] Formulas for the Walsh coefficients of smooth functions and their application to bounds on the Walsh coefficients - [http://arxiv.org/abs/2406.18700v4] Structure of sparse Boolean functions over Abelian groups, and its application to testing - [http://arxiv.org/abs/2205.09222v2] On the Walsh and Fourier-Hadamard Supports of Boolean Functions From a Quantum Viewpoint - [http://arxiv.org/abs/1908.05361v2] Placing quantified variants of 3-SAT and Not-All-Equal 3-SAT in the polynomial hierarchy - [http://arxiv.org/abs/1402.4455v1] Symbiosis of Search and Heuristics for Random 3-SAT - [http://arxiv.org/abs/1505.03340v2] HordeSat: A Massively Parallel Portfolio SAT Solver - [http://arxiv.org/abs/1908.01624v1] Learned Clause Minimization in Parallel SAT Solvers - [http://arxiv.org/abs/quant-ph/0305179v3] Polynomial Degree and Lower Bounds in Quantum Complexity: Collision and Element Distinctness with Small Range

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
import sys
from collections import defaultdict

def is_unsat(F):
    n = max(abs(l) for clause in F for l in clause)
    assignment = [None] * (n + 1)

    def propagate():
        changed = True
        while changed:
            changed = False
```

```
            for i in range(1, n + 1):
                if assignment[i] is None:
                    pos = any(l > 0 and assignment[l] == 1 for l in clause)
                    neg = any(l < 0 and assignment[-l] == -1 for l in clause)
                    if pos and not neg:
                        assignment[i] = 1
                        changed = True
                    elif not pos and neg:
                        assignment[i] = -1
                        changed = True

    propagate()

    for clause in F:
        if all(assignment[l] == (l > 0) * 2 - 1 for l in clause):
            return False
    return True

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [8, 10, 12, 14]
    m_values = [math.floor(4.5 * n) for n in n_values]

    results = []
    for n, m in zip(n_values, m_values):
        count_unsat = 0
        count_random = 0
        total_leaves = 0

        for _ in range(30):
            F = []
            while True:
                clause = [random.randint(-n, -1), random.randint(1, n)]
                if len(set(clause)) == 2 and all(abs(l) not in F for l in clause):
                    F.append(tuple(sorted(clause)))
                    break
            if is_unsat(F):
                count_unsat += 1
                leaves = 0
                g_F = [0] * (1 << n)
                for i in range(1 << n):
                    assignment = [((i >> j) & 1) * 2 - 1 for j in range(n)]
                    if is_unsat([list(clause)] + F, assignment):
                        leaves += 1
                        g_F[i] = 1
                total_leaves += leaves
                k = int(math.log2(m + 1))
                T_k = sum(g_F[i]**2 for i in range(1 << n) if bin(i).count('1') > k)
                log_L = math.log2(leaves + 1)
                max_hat_g_emptyset = max(abs(sum(g_F[i] * (1 << j) for i in range(1 << n))) for j
                conjecture_holds = log_L <= 4 * (1 + T_k * 2**n) / max(1, max_hat_g_emptyset * 2**
                ratio = log_L / (1 + T_k * 2**n / max(1, max_hat_g_emptyset * 2**n))
                results.append({
                    "metric_name": "log2(L+1)",
                    "metric_value": log_L,
                    "instances_tested": 1,
```

```
                    "conjecture_holds": conjecture_holds,
                    "counterexample": "" if conjecture_holds else f"Ratio out of bounds: {ratio}"
                })

        for _ in range(30):
            F = []
            while len(F) < m:
                clause = [random.randint(-n, -1), random.randint(1, n)]
                if len(set(clause)) == 2 and all(abs(l) not in F for l in clause):
                    F.append(tuple(sorted(clause)))
            if is_unsat(F):
                count_random += 1
                leaves = 0
                g_F = [0] * (1 << n)
                for i in range(1 << n):
                    assignment = [((i >> j) & 1) * 2 - 1 for j in range(n)]
                    if is_unsat([list(clause)] + F, assignment):
                        leaves += 1
                        g_F[i] = 1
                total_leaves += leaves
                k = int(math.log2(m + 1))
                T_k = sum(g_F[i]**2 for i in range(1 << n) if bin(i).count('1') > k)
                log_L = math.log2(leaves + 1)
                max_hat_g_emptyset = max(abs(sum(g_F[i] * (1 << j) for i in range(1 << n))) for j
                ratio = log_L / (1 + T_k * 2**n / max(1, max_hat_g_emptyset * 2**n))
                results.append({
                    "metric_name": "log2(L+1)",
                    "metric_value": log_L,
                    "instances_tested": 1,
                    "conjecture_holds": ratio >= 0.05 and ratio <= 4,
                    "counterexample": "" if ratio >= 0.05 and ratio <= 4 else f"Ratio out of bound
                })

    mean_log_L = sum(result["metric_value"] for result in results) / len(results)
    std_log_L = math.sqrt(sum((result["metric_value"] - mean_log_L)**2 for result in results) / le
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    return {
        "seed": seed,
        "mean_log_L": mean_log_L,
        "std_log_L": std_log_L,
        "support_fraction": support_fraction
    }

if __name__ == "__main__":
    seeds = list(map(int, sys.argv[1:])) if len(sys.argv) > 1 else [11, 23, 37, 53, 71]

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")

    mean_log_L = sum(result["mean_log_L"] for result in results) / len(results)
    std_log_L = math.sqrt(sum((result["mean_log_L"] - mean_log_L)**2 for result in results) / len(
    support_fraction = sum(1 for result in results if result["support_fraction"] >= 0.6) / len(res

    if all(result["support_fraction"] >= 0.6 for result in results):
```

```
        print(f"RESULT: SUPPORTED mean={mean_log_L} std={std_log_L} support_fraction={support_frac
    elif sum(1 for result in results if result["support_fraction"] < 0.6) / len(results) <= 0.2:
        print("RESULT: FALSIFIED counterexample=\"not enough support\" first_failing_seed=NA")
    else:
        print(f"RESULT: INCONCLUSIVE not enough evidence to confirm or refute the conjecture")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7e919bf9.py", line 129, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7e919bf9.py", line 61, in run_trial
    if is_unsat(F):
       ^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7e919bf9.py", line 36, in is_unsat
    propagate()
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7e919bf9.py", line 27, in propagate
    pos = any(l > 0 and assignment[l] == 1 for l in clause)
                                                    ^^^^^^
NameError: cannot access free variable 'clause' where it is not associated with a value in enclosing sc
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test crashed with a NameError in the propagate() function before producing any data, so no support or falsification evidence was generated. Per rule 2, a crash yields INCONCLUSIVE. | next: Fix the scoping bug in propagate() (capture `clause` properly in the generator expression, e.g., bind it as a loop variable or pass it explicitly) and rerun the test.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	20534
2	preregistration	claude_max	opus	0	0	7120
3	novelty	claude_max	opus	0	0	3479
4	novelty	claude_max	opus	0	0	7913

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18450
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17248
7	judge	claude_max	opus	0	0	4861

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 79605 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/9edcbf300e22.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/9edcbf300e22.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/9edcbf300e22.tar.gz` (if generated)